Laboratório 4: Visualização de Dados com plotnine

Jhuan Marcos Nunes Solis Silva - 266349

```
# Disponibiliza plotnine e pandas no ambiente Python
reticulate::py_require(c("plotnine", "pandas"))
```

Carregamento dos pacotes

```
import numpy as np
import pandas as pd
from plotnine import *
```

Questão 1 — Importação dos dados

O arquivo `diamonds.csv` é lido da pasta do projeto. Algumas versões do arquivo trazem uma coluna de índice sem nome, que é descartada. As variáveis categóricas são convertidas para fatores ordenados, do pior para o melhor nível, para que legendas e painéis apareçam numa ordem que faça sentido.

```
diamonds = pd.read_csv("diamonds.csv.gz")
diamonds = diamonds.loc[:, ~diamonds.columns.str.startswith("Unnamed")]

ordem_cut     = ["Fair", "Good", "Very Good", "Premium", "Ideal"]
ordem_color   = ["D", "E", "F", "G", "H", "I", "J"]          # melhor -> pior
ordem_clarity = ["I1", "SI2", "SI1", "VS2", "VS1", "VVS2", "VVS1", "IF"]  # pior
-> melhor

diamonds["cut"]     = pd.Categorical(diamonds["cut"], ordem_cut, ordered=True)
diamonds["color"]         =   pd.Categorical(diamonds["color"],   ordem_color,
ordered=True)
diamonds["clarity"]   =   pd.Categorical(diamonds["clarity"],   ordem_clarity,
ordered=True)

print(diamonds.shape)
```

```
(53940, 10)
```

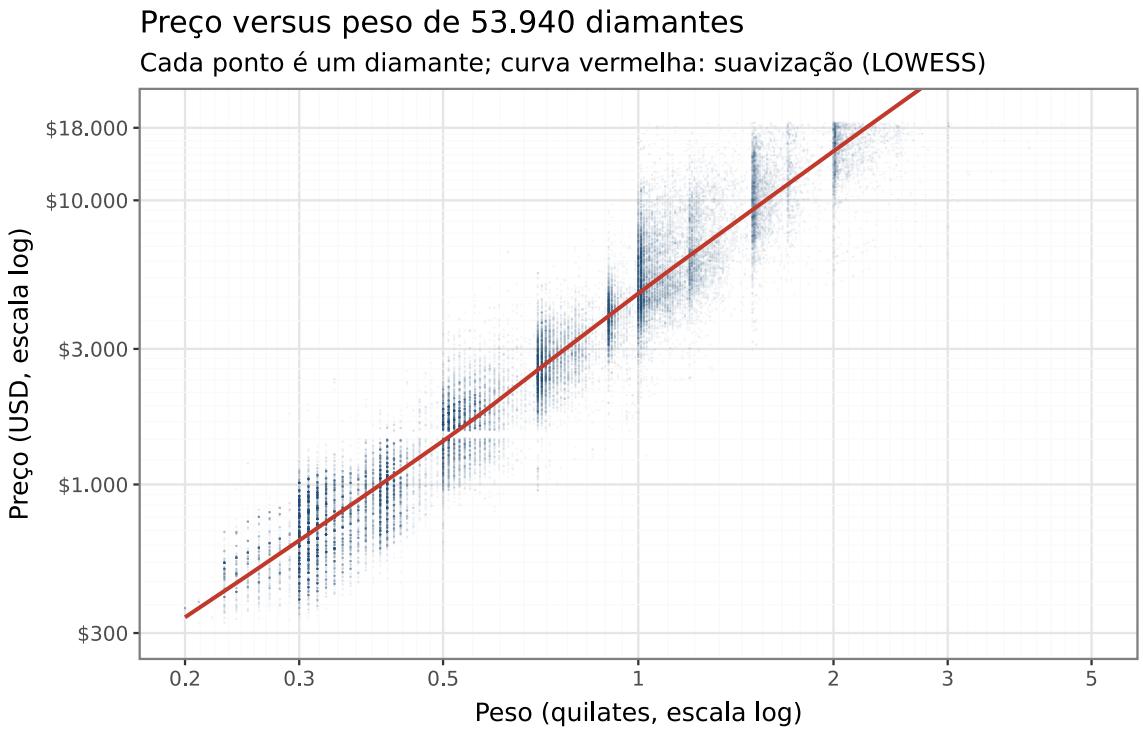
```
diamonds.head()
```

```
   carat      cut color clarity  depth  table  price     x     y     z
0   0.23    Ideal     E     SI2   61.5   55.0    326  3.95  3.98  2.43
1   0.21  Premium     E     SI1   59.8   61.0    326  3.89  3.84  2.31
2   0.23     Good     E     VS1   56.9   65.0    327  4.05  4.07  2.31
3   0.29  Premium     I     VS2   62.4   58.0    334  4.20  4.23  2.63
4   0.31     Good     J     SI2   63.3   58.0    335  4.34  4.35  2.75
```

```
# Elementos visuais reaproveitados em todos os gráficos
tema       = theme_bw(base_size=11)
quebras_ct = [0.2, 0.3, 0.5, 1, 2, 3, 5]
quebras_p  = [300, 1000, 3000, 10000, 18000]
rotulos_p  = ["$300", "$1.000", "$3.000", "$10.000", "$18.000"]
cores_cut  = ["#440154", "#3b528b", "#21918c", "#5ec962", "#d4a017"]
```

Questão 2 — Associação entre peso e preço

```
p1 = (
    ggplot(diamonds, aes("carat", "price"))
    + geom_point(alpha=0.05, size=0.6, stroke=0, color="#1f4e79")
    + geom_smooth(method="lowess", se=False, color="#c0392b", size=1)
    + scale_x_log10(breaks=quebras_ct)
    + scale_y_log10(breaks=quebras_p, labels=rotulos_p)
    + coord_cartesian(ylim=(300, 20000))
    + labs(x="Peso (quilates, escala log)", y="Preço (USD, escala log)",
           title="Preço versus peso de 53.940 diamantes",
        subtitle="Cada ponto é um diamante; curva vermelha: suavização (LOWESS)")
    + tema
    + theme(figure_size=(7, 4.5))
)
_ = p1.draw(show=True)
```




```
r_original = diamonds["carat"].corr(diamonds["price"])
r_log      = np.log10(diamonds["carat"]).corr(np.log10(diamonds["price"]))
inclinacao              =              np.polyfit(np.log10(diamonds["carat"]),
np.log10(diamonds["price"]), 1)[0]
print(f"Correlação (escala original): {r_original:.3f}")
print(f"Correlação (escala log-log):  {r_log:.3f}")
print(f"Inclinação log-log: {inclinacao:.2f}   ->  dobrar o peso multiplica o
preço por {2**inclinacao:.1f}")
```

Cuidados tomados para um gráfico de boa qualidade. (i) Usei escala logarítmica nos dois eixos: na escala original, os pontos ficam espremidos no canto inferior esquerdo e a relação aparece como uma curva cuja variabilidade cresce com o peso; em log-log a relação vira praticamente uma reta e a dispersão fica homogênea. (ii) Como são quase 54 mil pontos, usei pontos pequenos, sem borda e com transparência alta (`alpha=0.05`) para reduzir a sobreposição e deixar visível onde os dados se concentram. (iii) Acrescentei uma curva de suavização para destacar a tendência sem impor uma forma. (iv) Os eixos têm marcações em valores "redondos", rótulos em dólares e títulos que informam a unidade e a escala, e o gráfico tem título e subtítulo explicando o que se vê.

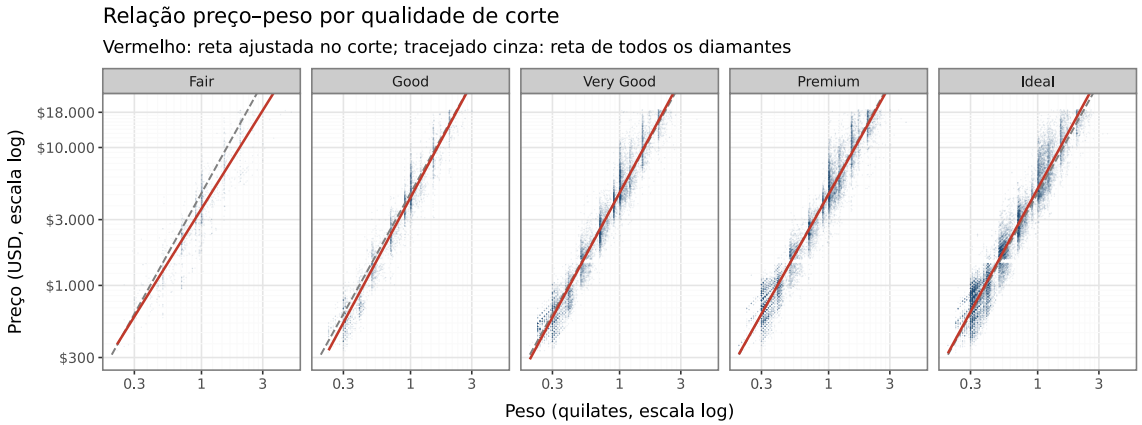
Achados. Peso e preço têm associação positiva muito forte (correlação de 0,92 na escala original e de 0,97 após o logaritmo). Em escala log-log a relação é aproximadamente linear, com inclinação de cerca de 1,7: dobrar o peso de um diamante multiplica seu preço, em média, por aproximadamente 3,2, ou seja, o preço cresce mais rápido que o peso. Dois detalhes chamam atenção. Os pontos formam faixas verticais em pesos "redondos" (0,3; 0,5; 0,7; 1; 1,5; 2 quilates), indicando que as

pedras costumam ser lapidadas para atingir esses valores. E, acima de cerca de 2 quilates, os preços se acumulam perto do teto de US$ 18.823 do conjunto de dados, de modo que a tendência nessa faixa se apoia em poucas observações e deve ser lida com cautela. Também há bastante variação vertical para um mesmo peso, sinal de que outras características influenciam o preço.

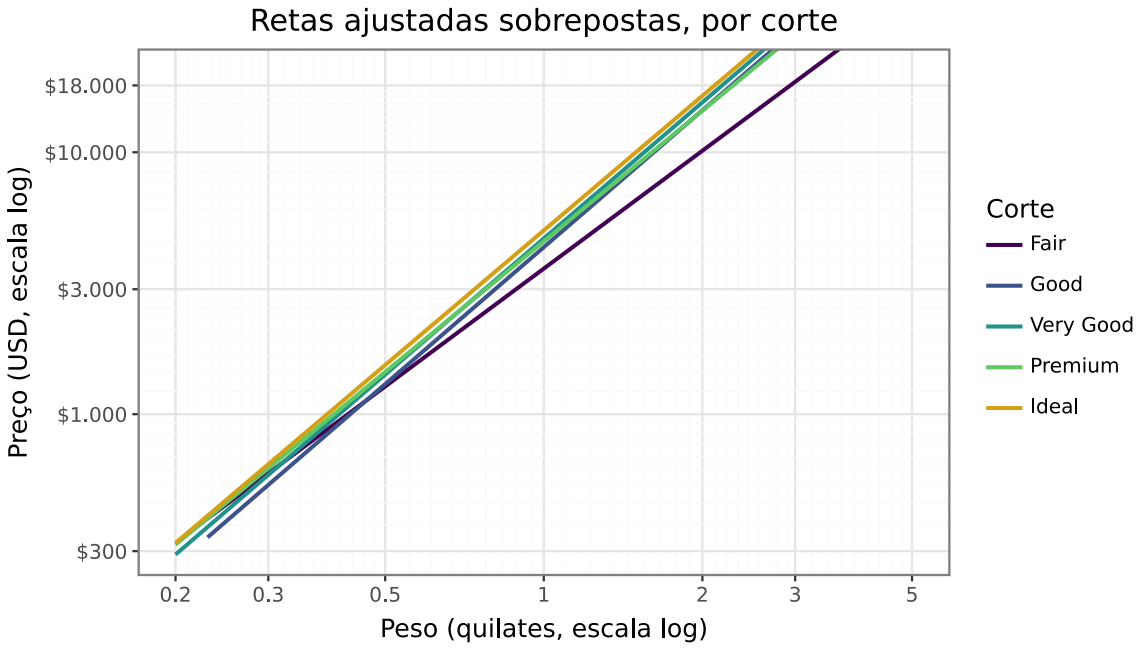
Questão 3 — A relação é a mesma para os diferentes cortes?

```
# Reta ajustada com todos os diamantes, repetida em cada painel como referência
coef_geral              =              np.polyfit(np.log10(diamonds["carat"]),
np.log10(diamonds["price"]), 1)
referencia = pd.DataFrame({"carat": np.geomspace(0.2, 5, 50)})
referencia["price"]        =        10        **        np.polyval(coef_geral,
np.log10(referencia["carat"]))

p2 = (
    ggplot(diamonds, aes("carat", "price"))
    + geom_point(alpha=0.08, size=0.5, stroke=0, color="#1f4e79")
    + geom_line(data=referencia, linetype="dashed", color="gray", size=0.7)
    + geom_smooth(method="lm", se=False, color="#c0392b", size=0.9)
    + facet_wrap("~cut", nrow=1)
    + scale_x_log10(breaks=[0.3, 1, 3])
    + scale_y_log10(breaks=quebras_p, labels=rotulos_p)
    + coord_cartesian(ylim=(300, 20000))
    + labs(x="Peso (quilates, escala log)", y="Preço (USD, escala log)",
           title="Relação preço–peso por qualidade de corte",
           subtitle="Vermelho: reta ajustada no corte; tracejado cinza: reta de
todos os diamantes")
    + tema
    + theme(figure_size=(10, 3.8))
)
_ = p2.draw(show=True)
```



```
p3 = (
    ggplot(diamonds, aes("carat", "price", color="cut"))
    + geom_smooth(method="lm", se=False, size=1)
    + scale_x_log10(breaks=quebras_ct)
    + scale_y_log10(breaks=quebras_p, labels=rotulos_p)
    + coord_cartesian(ylim=(300, 20000))
    + scale_color_manual(values=cores_cut)
      + labs(x="Peso (quilates, escala log)", y="Preço (USD, escala log)",
color="Corte",
           title="Retas ajustadas sobrepostas, por corte")
    + tema
    + theme(figure_size=(7, 4))
)
_ = p3.draw(show=True)
```




```
def ajuste_log(g):
    a, b = np.polyfit(np.log10(g["carat"]), np.log10(g["price"]), 1)
    return pd.Series({"n": len(g),
                      "peso_mediano": g["carat"].median(),
                      "preco_mediano": g["price"].median(),
                      "inclinacao": round(a, 2),
                      "preco_1_quilate": round(10 ** b)})

diamonds.groupby("cut", observed=True)[["carat", "price"]].apply(ajuste_log)
```

Conclusão. A forma da relação é essencialmente a mesma nos cinco cortes: em todos os painéis os pontos seguem uma reta em escala log-log, com inclinações parecidas (entre 1,5 no corte *Fair* e cerca de 1,7 nos demais). O que muda é o nível: para um mesmo peso, cortes melhores são um pouco mais caros. Um diamante de 1 quilate custa em média cerca de US$ 3.600 no corte *Fair* e cerca de US$ 5.000 no corte *Ideal*, e as retas de *Good*, *Very Good* e *Premium* ficam entre essas duas e muito próximas entre si. O corte *Fair* é o que mais se afasta, com reta abaixo e menos inclinada que a de referência. Vale notar que, olhando só o preço mediano, os diamantes *Ideal* parecem os mais baratos (US$ 1.810 contra US$ 3.282 dos *Fair*); isso acontece porque eles são, em geral, bem menores (mediana de 0,54 contra 1,00 quilate). Comparar o preço sem controlar pelo peso levaria a uma conclusão errada, e é por isso que a visualização precisa mostrar o peso no eixo horizontal.

Questão 4 — Corte, cor e clareza ao mesmo tempo

São 5 cortes × 7 cores × 8 clarezas = 280 combinações possíveis. Antes de visualizar, contei quantos diamantes há em cada uma:

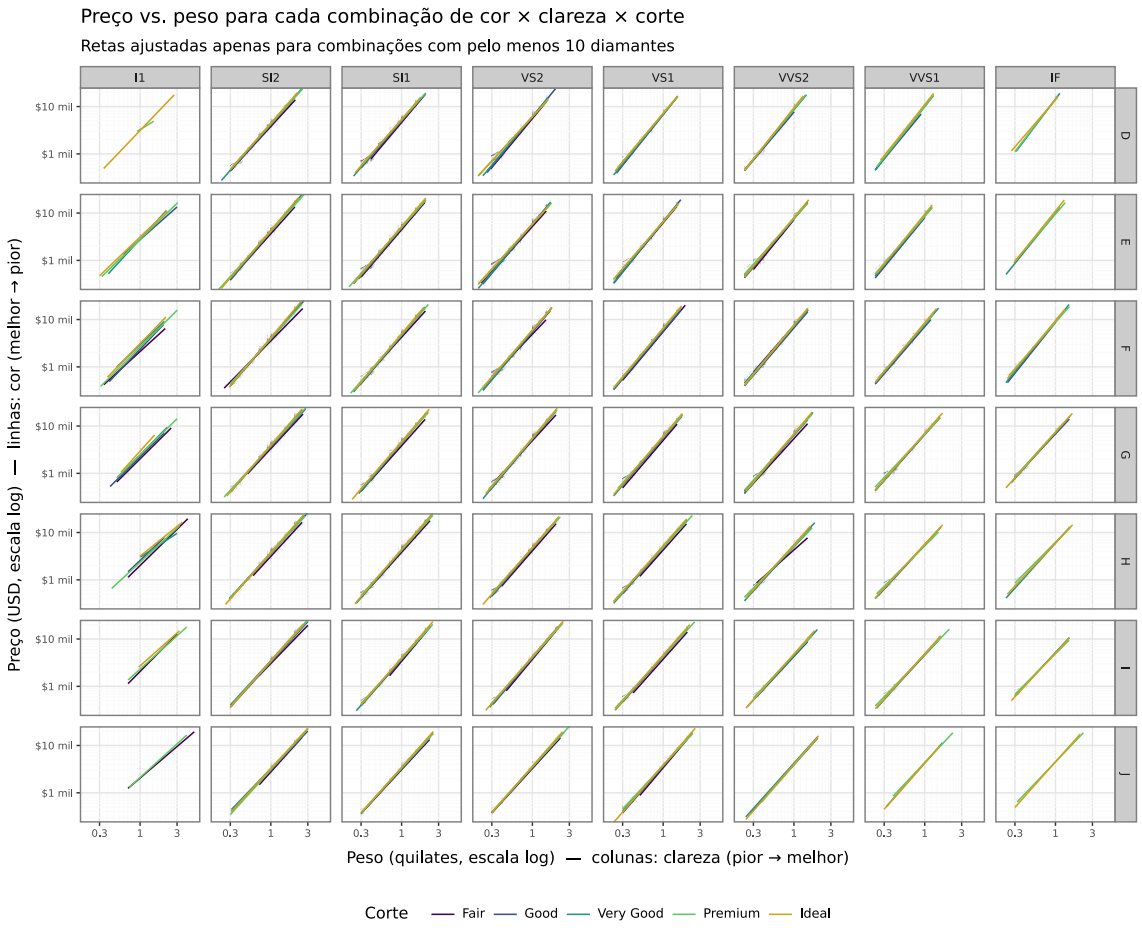
```
contagem = (diamonds.groupby(["cut", "color", "clarity"], observed=True)
            .size().rename("n").reset_index())
print("Combinações observadas:", len(contagem), "de", 5 * 7 * 8)
print("Combinações com menos de 10 diamantes:", (contagem["n"] < 10).sum())
contagem["n"].describe()
```

A primeira visualização usa uma grade de painéis (cor nas linhas, clareza nas colunas) e, dentro de cada painel, uma reta por corte. Retas só são desenhadas para combinações com pelo menos 10 diamantes.

```
dados_q4 = diamonds.merge(contagem, on=["cut", "color", "clarity"])

p4 = (
    ggplot(dados_q4, aes("carat", "price"))
    + geom_point(alpha=0.15, size=0.3, stroke=0, color="gray")
    + geom_smooth(data=dados_q4[dados_q4["n"] >= 10], mapping=aes(color="cut"),
                  method="lm", se=False, size=0.6)
    + facet_grid("color ~ clarity")
    + scale_x_log10(breaks=[0.3, 1, 3])
    + scale_y_log10(breaks=[1000, 10000], labels=["$1 mil", "$10 mil"])
    + coord_cartesian(ylim=(300, 20000))
    + scale_color_manual(values=cores_cut)
    + labs(x="Peso (quilates, escala log)  —  colunas: clareza (pior → melhor)",
           y="Preço (USD, escala log)  —  linhas: cor (melhor → pior)",
           color="Corte",
           title="Preço vs. peso para cada combinação de cor × clareza × corte",
             subtitle="Retas ajustadas apenas para combinações com pelo menos
10 diamantes")
```

```
    + tema
    + theme(figure_size=(11, 9), legend_position="bottom",
            strip_text=element_text(size=8), axis_text=element_text(size=7))
)
_ = p4.draw(show=True)
```

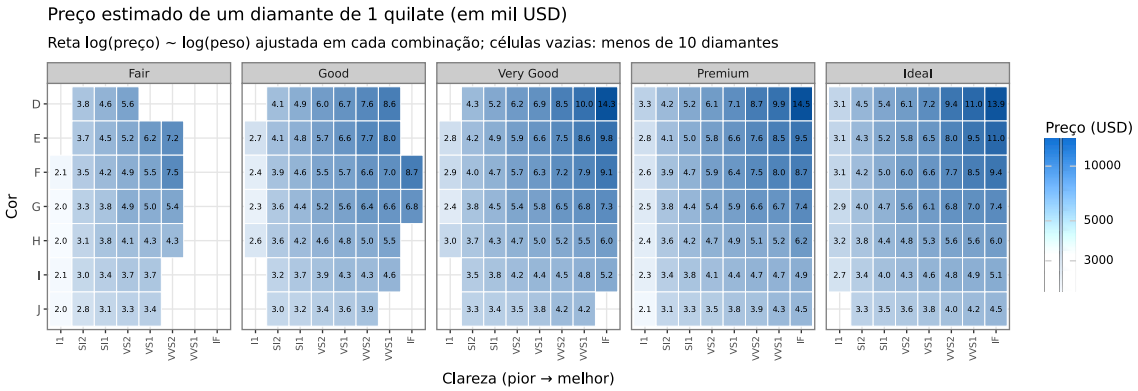


Como as retas ficam muito sobrepostas, a segunda visualização resume cada combinação em dois números, obtidos ajustando a reta log(preço) ~ log(peso) dentro dela: a inclinação e o preço previsto para um diamante de 1 quilate. O mapa de calor mostra este último.

```
def ajuste_combinacao(g):
    if len(g) < 10:
      return pd.Series({"inclinacao": np.nan, "preco_1ct": np.nan, "n": len(g)})
    a, b = np.polyfit(np.log10(g["carat"]), np.log10(g["price"]), 1)
    return pd.Series({"inclinacao": a, "preco_1ct": 10 ** b, "n": len(g)})
```

```
coef = (diamonds.groupby(["cut", "color", "clarity"], observed=True)[["carat",
"price"]]
        .apply(ajuste_combinacao).reset_index()
        .dropna())
coef["rotulo"] = (coef["preco_1ct"] / 1000).round(1).astype(str)
coef[["inclinacao", "preco_1ct"]].describe().round(2)
```

```
p5 = (
    ggplot(coef, aes("clarity", "color", fill="preco_1ct"))
    + geom_tile(color="white")
    + geom_text(aes(label="rotulo"), size=6)
    + facet_wrap("~cut", nrow=1)
    + scale_y_discrete(limits=ordem_color[::-1])
        +  scale_fill_gradient(low="#f7fbff",  high="#08519c",  trans="log10",
name="Preço (USD)")
    + labs(x="Clareza (pior → melhor)", y="Cor",
           title="Preço estimado de um diamante de 1 quilate (em mil USD)",
           subtitle="Reta log(preço) ~ log(peso) ajustada em cada combinação; "
                    "células vazias: menos de 10 diamantes")
    + tema
    + theme(figure_size=(12, 4.2), axis_text_x=element_text(angle=90, size=7))
)
_ = p5.draw(show=True)
```



Dificuldades encontradas. A principal é a quantidade de combinações: com 280 grupos não dá para usar uma cor ou um símbolo para cada um, então foi preciso combinar painéis (cor × clareza) com cores (corte), o que gera uma figura grande, com painéis pequenos e retas sobrepostas difíceis de distinguir. A segunda é o desequilíbrio entre os grupos: das 280 combinações, 4 não têm nenhum diamante e outras 30 têm menos de 10, algumas com um ou dois. Ajustar uma reta nesses casos não faz sentido, então elas foram excluídas, o que deixa "buracos" na figura (por exemplo, o corte *Fair* quase não tem diamantes de clareza VVS1 ou IF). A terceira é a sobreposição de pontos,

resolvida com escala log, transparência e pontos cinza ao fundo. Por fim, foi necessário reordenar manualmente as categorias, pois por padrão elas apareceriam em ordem alfabética, e não do pior para o melhor. Para contornar a poluição visual, a segunda figura troca os dados brutos por um resumo (agregação por grupo), o que torna as diferenças de nível fáceis de ler.

A relação é a mesma para diferentes configurações? A *forma* da relação é a mesma: em praticamente todos os painéis o preço cresce como uma reta em escala log-log, com inclinação mediana de cerca de 1,86 (cerca de 80% das combinações entre 1,7 e 2,0). As exceções com inclinações bem menores estão quase todas na clareza I1, justamente onde há poucos dados. O que muda bastante entre as configurações é o *nível* da reta. O preço estimado de 1 quilate vai de cerca de US$ 2.000 (cor J, clareza I1, corte *Fair*) a cerca de US$ 14.500 (cor D, clareza IF). Clareza e cor são as características que mais deslocam a reta: em qualquer corte o mapa escurece da esquerda para a direita (clareza melhor) e de baixo para cima (cor melhor). Dentro de uma mesma combinação de cor e clareza, o corte tem efeito pequeno: as retas dos cinco cortes quase se sobrepõem. Um detalhe interessante é que a inclinação dentro dos grupos (cerca de 1,86) é maior que a inclinação geral da Questão 2 (cerca de 1,68). Isso ocorre porque diamantes grandes tendem a ter cor e clareza piores; quando se compara apenas diamantes de mesma qualidade, o peso pesa ainda mais no preço.

Data e hora da execução

```
print(format(Sys.time(), "%d/%m/%Y %H:%M:%S %Z", tz = "America/Sao_Paulo"))
```

```
[1] "19/10/2026 17:30:02 -03"
```